TIPOS DE DATOS ABSTRACTOS

Guía Práctica

Contenido Sinóptico

Especificación Operacional (Pre/Post) de un TDA. Implementación de un TDA. Estructuras lineales: Listas, Pilas y Colas.

Especificación e Implementación de un TDA

1. Se desea definir el TDA Matriz[Elemento]. Una matriz es una estructura de la siguiente forma:

$$M = \begin{pmatrix} a_{11} & a_{12} & \dots & a_{1m} \\ a_{21} & a_{22} & \dots & a_{2m} \\ \dots & \dots & \dots & \dots \\ a_{n1} & a_{2n} & \dots & a_{nm} \end{pmatrix}$$

Donde:
$(a_{ij} \in \text{Elemento}) \wedge (i, j \in \text{N}) \wedge (1 \leq i \leq n) \wedge (1 \leq j \leq m)$

Sus operaciones son las siguientes:

- mat_crear: Crea una matriz con un número específico de filas u columnas.
- mat_asigna: Asigna un valor a una posición de la matriz.
- mat_valor: Retorna el valor de una posición de la matriz.
- mat_filas: Retorna el número de filas de la matriz.
- mat_cols: Retorna el número de columnas de la matriz.

Se desea que:

a) Elabore la especificación del TDA Matriz.
b) Desarrolle una implementación del TDA Matriz.

2. El TDA Matriz[Real] es un caso particular del TDA Matriz[Elemento], cuando Elemento = Real.

a) Extienda la especificación del TDA Matriz[Real] con las siguientes operaciones:
   - mat_suma: Retorna el resultado de sumar dos matrices.
   - mat_resta: Retorna el resultado de restar dos matrices.
   - mat_multiplicacion: Retorna el resultado de multiplicar dos matrices.

b) Realice la implementación de dichas operaciones.

3. Se desea definir el tipo Palabra, cuyos objetos son palabras de un texto.
Una palabra es una secuencia de caracteres de la forma:

$$P = \text{“}c_1c_2c_3...c_n\text{”}$$

Donde cada c_i es un caracter perteneciente al alfabeto.

Las operaciones del tipo Palabra, escritas informalmente, son las siguientes:

- palabra_vacía: Genera una palabra vacía. P = “”
- es_vacía: Informa si una palabra dada es vacía.
- longitud_palabra: Retorna la longitud de una palabra.
- consultar_letra: Retorna la *i*-ésima letra de una palabra.
- agregar_letra: Agrega una letra al final de la palabra.
- son_iguales: Informa si dos palabras son iguales.

Se desea que:

a) Elabore la especificación del TDA Palabra.
b) Desarrolle una implementación del TDA Palabra.

4. Se desea definir el tipo RelojMilitar. Un reloj militar se define como sigue:

$$R = horas{:}minutos{:}segundos$$

Donde:
$(horas, minutos, segundos \in \mathbb{N}) \wedge (0 \leq horas \leq 23) \wedge (0 \leq minutos \leq 59) \wedge (0 \leq segundos \leq 59)$

Sus operaciones son las siguientes:

- crear: Crea un nuevo reloj militar en la hora 00:00:00.
- hora: Retorna la hora del reloj.
- minuto: Retorna el minuto del reloj.
- segundo: Retorna el segundo del reloj.
- siguiente_hora: Aumenta la hora del reloj al siguiente valor válido.
- siguiente_minuto: Aumenta el minuto del reloj al siguiente valor válido.
- siguiente_segundo: Aumenta el segundo del reloj al siguiente valor válido.
- avanzar: Aumenta un segundo del reloj. En caso de que el nuevo segundo sea 0, se debe aumentar un minuto; en caso de que el nuevo minuto sea 0, se debe aumentar una hora.

Se desea que:

a) Elabore la especificación del TDA RelojMilitar.
b) Desarrolle una implementación del TDA RelojMilitar.

5. Se desea definir el tipo Fecha , el cual está definido por tres componentes:

dia/mes/año

Donde:

- Los días son valores enteros entre 1 y 31, dependiendo del mes:
  - Los meses de Enero, Marzo, Mayo, Julio, Agosto, Octubre y Diciembre tienen 31 días.
  - Los meses de Abril, Junio, Septiembre y Noviembre tienen 30 días.
  - El mes de febrero tiene 28 días, y 29 en los años bisiestos.
- Es un año bisiesto aquel divisible entre 4, excepto aquellos que son divisibles entre 100 pero no entre 400.
- Los meses son valores enteros entre 1 y 12.
- Los años son valores enteros mayores que 0 (Tomando en cuenta desde el año 1 d.C.).

Las operaciones del tipo Fecha, descritas informalmente, son las siguientes:

- crear_fecha: Genera una Fecha
- obtener_día: Retorna el día de una Fecha
- obtener_mes: Retorna el mes de una Fecha
- obtener_año: Retorna el año de una Fecha
- es_bisiesto: Informa si el año de una Fecha es bisiesto
- es_menor: Informa si una Fecha es menor que otra
- son_iguales: Informa si dos Fechas son iguales
- es_mayor: Informa si una Fecha es mayor que otra

Se desea que:

a) Elabore la especificación del TDA Fecha.
b) Desarrolle una implementación del TDA Fecha.

6. Se desea definir el TDA Polinomio. Un polinomio es un objeto matemático con la siguiente forma:

$$P = c_n x^n + c_{n-1} x^{n-1} + \ldots + c_2 x^2 + c_1 x^1 + c_0 x^0$$

Donde:
$(i \in \mathrm{Z}) \wedge (n \in \mathrm{Z}) \wedge (0 \leq i \leq n) \wedge (c_i \in \mathrm{R}) \wedge (x_i \in \mathrm{R})$
c_i, es el coeficiente del i-ésimo término del polinomio.
x^i es una variable elevada a la i-ésima potencia.

Sus operaciones son las siguientes:

- crear_polinomio: Crea un nuevo polinomio. Este debe ser por defecto nulo.
- grado: Consulta el grado del polinomio.
- coeficiente: Consulta el coeficiente del monomio de grado *i*.
- asignar_coeficiente: Modifica el coeficiente del monomio de grado *i*.
- evaluar_polinomio: Dado un valor para *x*, da el resultado de evaluar el polinomio.
- sumar_polinomio: Retorna el resultado de sumar dos polinomios.
- restar_polinomio: Retorna el resultado de restar dos polinomios.

Se desea que:

a) Elabore la especificación del TDA Polinomio.
b) Desarrolle una implementación del TDA Polinomio.

7. Un texto puede verse, intuitivamente, como una colección ordenada de palabras. Las palabras son los valores del TDA Palabra. Las operaciones del tipo Texto, descritas informalmente, son las siguientes:

- texto_vacío: Produce el texto vacío
- obtener_Palabra: Extrae la primera palabra del texto
- es_vacío: Informa si un texto dado es vacío
- eliminar_palabra: Elimina la primera palabra del texto
- agregar_palabra: Agrega una palabra al final del texto

Se desea que:

a) Elabore la especificación del TDA Texto.
b) Desarrolle una implementación del TDA Texto.

8. Un libro se puede ver como un objeto abstracto que está formado por los siguientes elementos:

- ISBN (*International Standard Book Number*: Es un código único de 13 dígitos que permite la identificación de un libro. Ejemplo: 978-3-16-148410-0, por sencillez se eliminan los guiones y queda 9783161484100).
- Autores (Cada uno tiene su nombre y apellido. El autor principal es el primero nombrado).
- Título.
- Año de publicación.
- Edición.
- Editorial.

Además cuenta con las siguientes operaciones:

- crear_libro: Dados todos los datos del libro, crea un nuevo libro.
- agregar_autor: Agrega un autor al libro.
- eliminar_autor: Elimina un autor del libro. Se debe tomar en cuenta que no es posible eliminar el autor principal.
- consultar_isbn: Consulta el ISBN del libro.
- consultar_autor: Consulta un autor del libro.
- consultar_titulo: Consulta el título del libro.
- consultar_anio: Consulta el año de publicación del libro.
- consultar_edicion: Consulta la edición del libro.
- consultar_editorial: Consulta la editorial del libro.
- modificar_isbn: Modifica el ISBN del libro.
- modificar_autor: Modifica un autor del libro.
- modificar_titulo: Modifica el título del libro.
- modificar_anio: Modifica el año de publicación del libro.
- modificar_edicion: Modifica la edición del libro.

- modificar_editorial: Modifica la editorial del libro.
- imprimir_referencia: Imprime una referencia al libro en el siguiente formato: $Apellido_1$, $Nombre_1$; …; $Apellido_n$, $Nombre_n$. (Año). Título (Edición). Editorial.

Se desea que:

a) Elabore la especificación del TDA Libro.
b) Desarrolle una implementación del TDA Libro.

9. Una biblioteca es una institución cuya finalidad consiste en la adquisición, conservación, estudio y exposición de libros. En una biblioteca los libros se encuentran organizados de acuerdo a diferentes criterios, en este caso, se toma en cuenta el autor del libro. Una biblioteca debe ser actualizada frecuentemente, por lo cual puede adquirir libros nuevos. Además en caso de que un libro se encuentre muy deteriorado, puede descartarlo.

Ejercicios de Listas, Pilas y Colas

10. Basándose en las operaciones del TDA Lista elabore algoritmos para resolver cada uno de los siguientes problemas:

a) Concatenar dos listas.
b) Invertir una lista.
c) Eliminar de la lista todos los elementos iguales a un valor X.
d) Combinar dos listas ordenadas en una sola ordenada.
e) Formar una lista que sea la intersección de dos listas dadas.
f) Verificar si una lista está ordenada.

11. Realice un algoritmo que convierta una expresión escrita en notación infija a postfija.

Consideraciones:

- La expresión viene dada por una cadena de caracteres y está bien formada.
- Utilice el TDA Pila.

Conversión de una expresión infija a postfija:

1. Repetir hasta que no haya caracteres en la expresión de entrada:
   1.1. Leer un caracter de la expresión.
   1.2. Si es un operando se coloca en la expresión postfija de salida.
   1.3. Si es un operador:
      1.3.1. Mientras la pila no sea vacía y su tope sea un operador de precedencia mayor o igual al operador leído y diferente a un paréntesis izquierdo:
      a. Extraer el tope de la pila y colocarlo en la expresión postfija de salida.
      1.3.2. Insertar el operador leído en la pila.
   1.4. Si es un paréntesis:
      1.4.1. Si es izquierdo se mete en la pila.
      1.4.2. Si es derecho se sacan y se colocan en la expresión postfija de salida todos los operadores de la pila hasta llegar a un paréntesis izquierdo, que se elimina.
2. Si quedan elementos en la pila pasarlos a la expresión postfija de salida.

12. Elabore un algoritmo que acepte como entrada una expresión aritmética en notación postfija y realice la evaluación de dicha expresión. Elija cuál de los TDAs estudiados (Lista, Pila, Cola) permite representar la expresión y evaluarla en términos de sus operaciones primitivas.

Consideraciones:

- La expresión aritmética viene dada en notación postfija.
- La expresión de entrada está almacenada en una lista de cadenas de caracteres.
- Dispone de las siguientes operaciones auxiliares:
  - es_numero(Cadena c): booleano, dada una cadena de caracteres c, determina si es un número o no.
  - convertir(Cadena c): real, dada una cadena de caracteres c que representa un número, retorna el equivalente numérico.

Ejemplo: 6 2 4 – * $\rightarrow$ (2 – 4) * 6 = –12

13. Dada una expresión aritmética que contenga paréntesis, corchetes y llaves, elabore un algoritmo que verifique si los anidamientos son correctos.

Ejemplos:

Expresión Correcta	{X + [Y – (A + B)] * C – [(D + E) – F]} / (J – (K – (M – N)))
Expresión Incorrecta	{X + [Y – (A + B)] * C –](D + E) – F]} / (J – (K – (M – N))

14. Desarrolle la implementación de las operaciones del TDA Cola usando como representación el TDA Pila y utilizando las operaciones definidas para este tipo de dato abstracto.

15. En un sistema basado en prioridades, los clientes son atendidos mediante una política según la cual los clientes que llegan hacen una cola de acuerdo a una prioridad que tienen asociada (existen M prioridades). Siempre es atendido el cliente que, teniendo mayor prioridad, haya llegado primero. Este sistema está representado por el TDA ColaP (Cola con Prioridades). La especificación operacional del TDA ColaP es la siguiente:

TDA ColaP [Elemento]

Sintaxis

$vacia_{CP}$:		$\rightarrow$	ColaP
es_vacia_{CP}:	ColaP	$\rightarrow$	lógico
$Encolar_{CP}$:	ColaP × Elemento × Natural	$\rightarrow$	ColaP
$desencolar_{CP}$:	ColaP	$\rightarrow$	ColaP
$frente_{CP}$:	ColaP	$\rightarrow$	Elemento

Dominio

D = {ColaP, Elemento, Natural, lógico}

Semántica

{Pre: }
func $vacia_{CP}$ (): ColaP
{Post: $vacia_{CP} \leftarrow < >$}

{Pre: }
func es_vacia_{CP} (ColaP C): lógico
{Post: $es_vacia_{CP} \leftarrow (C = < >)$}

{Pre: $1 \leq p \leq M$}
proc $encolar_{CP}$ (**ref** ColaP C; Elemento e; **entero** p)
{Post: Si $C = < [e_1, p_1], [e_2, p_2], \ldots, [e_k, p_k], [e_{k+1}, p_{k+1}], \ldots, [e_{n-1}, p_{n-1}], [e_n, p_n] >$ y $(p_k \geq p > p_{k+1})$ entonces luego de $encolar_{CP}$ $C = < [e_1, p_1], [e_2, p_2], \ldots, [e_k, p_k], [e, p], [e_{k+1}, p_{k+1}], \ldots, [e_{n-1}, p_{n-1}], [e_n, p_n] >$ }

{Pre: $\neg es_vacia_{CP}$ (C) }
proc $desencolar_{CP}$ (**ref** ColaP C)
{Post: Si $C = < [e_1, p_1], [e_2, p_2], \ldots, [e_{n-1}, p_{n-1}], [e_n, p_n] >$ entonces luego de $desencolar_{CP}$
$C = < [e_2, p_2], \ldots, [e_{n-1}, p_{n-1}], [e_n, p_n] >$ }

{Pre: $\neg es_vacia_{CP}$ (C) }
func $frente_{CP}$ (ColaP C): Elemento
{Post: Si $C = < [e_1, p_1], [e_2, p_2], \ldots, [e_{n-1}, p_{n-1}], [e_n, p_n] >$ entonces frente $\leftarrow e_1$ }

Implemente todas las operaciones del TDA ColaP [Elemento] en términos de las estructuras de datos seleccionadas para representar los objetos del TDA. Observación: Utilice los TDA estudiados en clase (Lista, Pila, Cola).